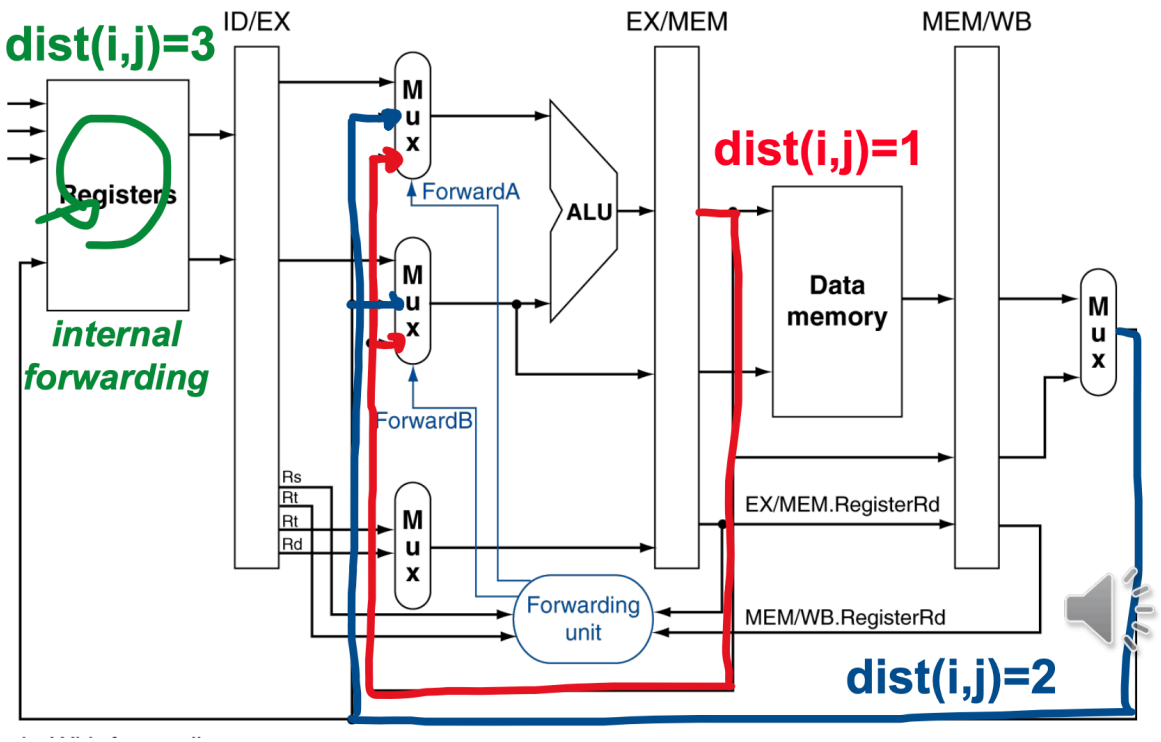
Lab 6 Report

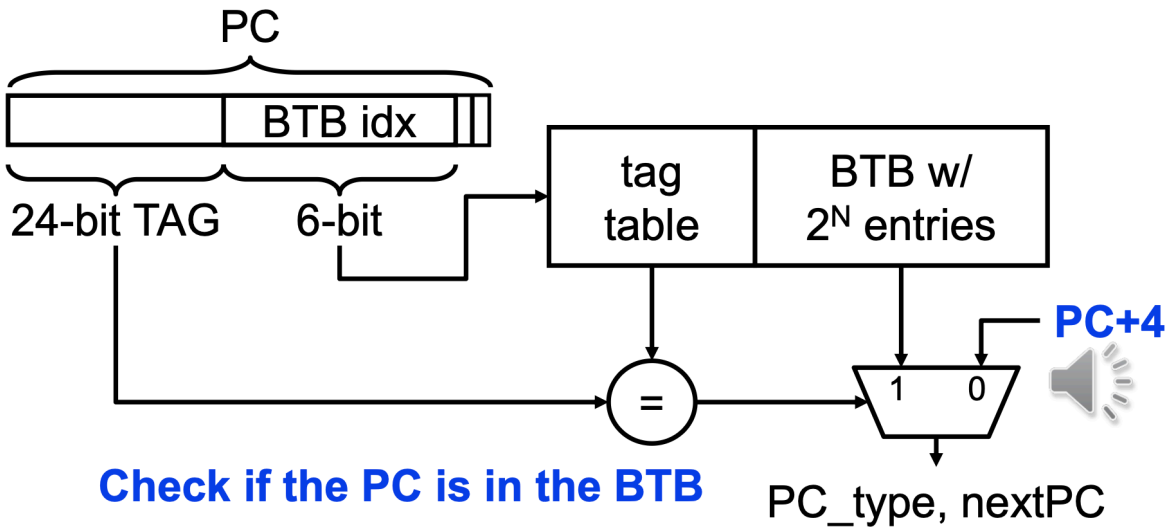
과제 개요

이번 과제는 HW05에서 구현하였던 pipeline CPU에 데이터 포워딩과 분기 예측 기능을 추가하는 과제다.



레지스터 파일에는 internal forwarding 로직을 추가하고, ALU 피연산자를 이전 ALU 연산 결과 또는 데이터 메모리에서 포워딩하는 기능을 추가해야 한다.

포워딩이 추가됨에 따라 필요 없어진 스톨을 hazard unit에서 제거하고, Load-Use hazard와 control dependency에 대한 스톨 로직을 추가해야 한다.



BTB는 태그 24-bit와 index 6-bit로 접근하는 방식이고, PHT는 2-bit saturation counter 방식을 사용한다.

구현 과정

Branch Predictor (`BP.v`)

가장 먼저 branch predictor를 구현하기 위해 `BP.v` 파일을 생성하였다.

```
module BP(
    input clk,
    input rst,
    input [31:0] PC, // current PC at IF stage

    output Pred_Taken,
    output [31:0] Pred_Target,

    // Ports for updating BP after branch resolution @ID st
age
    input Update_En,
    input [31:0] Update_PC,
    input [31:0] Update_Target,
    input Update_Taken,
    input Update_IsBranch,
```

```
    input Update_IsJump
);
```

입력으로 IF 단계의 현재 PC값 32-bit를 받아 분기 예측의 결과 `Pred_Taken`과 이동할 주소 `Pred_Target`을 출력하는 모듈이다. 추가로 branch resolution 이후 ID 단계에서 branch predictor를 업데이트하는 방식을 채택하여 업데이트를 위한 포트도 구현하였다.

```
        // BTB: 64 entries (6-bit index)
    reg [23:0] BTB_Tag [0:63];     // 24-bit Tag
    reg [31:0] BTB_Target [0:63];
    reg [1:0]  BTB_Type [0:63];    // 00: Empty, 01: Branc
h, 10: Jump

    // PHT: 256 entries (8-bit index), 2-bit Saturating Cou
nter
    reg [1:0] PHT [0:255];
```

태그 24-bit와 인덱스 6-bit로 구성된 64개의 entry를 가지는 BTB를 구현하였다. `BTB_Type`에 BTB entry가 비어있는지(00), branch인지(01), jump인지(10)를 나타내는 2-bit를 저장한다.

인덱스 8-bit로 구성된 256개의 entry를 가지는 2-bit saturation counter 기반 PHT도 구현하였다.

```
                if (rst) begin
            for (i = 0; i < 64; i = i + 1) begin
                BTB_Type[i] <= 2'b00;
            end
            for (i = 0; i < 256; i = i + 1) begin
                PHT[i] <= 2'b01; // Initialzie to 'Weakly N
ot Taken'
            end
        end
```

Reset 신호가 들어오면 `BTB_Type`을 모두 00으로 비워주고 PHT는 weakly not taken, 01로 초기화해주었다.

```
                else if (Update_En) begin
            // BTB update
```

```
            BTB_Tag[Update_PC[7:2]] <= Update_PC[31:8];
            BTB_Target[Update_PC[7:2]] <= Update_Target;
            BTB_Type[Update_PC[7:2]] <= Update_IsJump ? 2'b
10 : 2'b01;

            // PHT update
            if (Update_IsBranch) begin
                case (PHT[Update_PC[9:2]])
                    2'b00: PHT[Update_PC[9:2]] <= Update_Ta
ken ? 2'b01 : 2'b00;
                    2'b01: PHT[Update_PC[9:2]] <= Update_Ta
ken ? 2'b10 : 2'b00;
                    2'b10: PHT[Update_PC[9:2]] <= Update_Ta
ken ? 2'b11 : 2'b01;
                    2'b11: PHT[Update_PC[9:2]] <= Update_Ta
ken ? 2'b11 : 2'b10;
                endcase
            end
                end
```

Update 신호가 들어오면 먼저 BTB를 업데이트한다. 해당 PC값의 BTB entry의 태그, 목표 주소, 유형을 업데이트해준다. 다음으로 PHT는 branch인 경우에만 2-bit saturation counter를 업데이트한다.

```
        // Branch prediction logic
    wire [5:0] btb_idx = PC[7:2];
    wire [23:0] btb_tag = PC[31:8];
    wire btb_hit = (BTB_Type[btb_idx] != 2'b00) && (BTB_Tag
[btb_idx] == btb_tag);

    wire [1:0] pht_val = PHT[PC[9:2]];

    assign Pred_Target = BTB_Target[btb_idx];

    assign Pred_Taken = btb_hit && (BTB_Type[btb_idx] ==
2'b10 || (BTB_Type[btb_idx] == 2'b01 && pht_val >= 2'b10));
```

Branch prediction의 로직은 위와 같다. `btb_hit`이 나는 조건은 현재 BTB index의 `BTB_Type`이 empty가 아닌, 즉 branch 또는 jump이면서, 현재 PC의 태그(`btb_tag`) 값이 BTB index의 태그(`BTB_Tag[btb_idx]`)와 일치해야 한다.

추가로 PHT 2-bit saturation counter의 값을 가져와 taken 여부를 판단한다. Jump면 무조건 Taken으로 예측하고, branch면 PHT 값(`pht_val`)이 2(`2'b10`) 이상, 즉 weakly taken 또는 strongly taken일 때만 taken으로 예측하도록 설계하였다.

Hazard Unit 수정 (`HAZARD.v`)

다음으로 `HAZARD.v` 파일을 수정하였다. 데이터 포워딩을 지원하기 때문에 불필요한 data hazard 스톨은 제거하고, Load-use 스톨과 branch 명령어의 데이터 의존성에 대한 스톨 조건만 유지하였다.

```
                // 1. Load-Use Data Hazard
        if (ID_EX_MemRead && ID_EX_rd != 0 && (ID_EX_rd ==
IF_ID_rs || ID_EX_rd == IF_ID_rt)) begin
            PCWrite = 0;
            IF_ID_Write = 0;
            ID_EX_Flush = 1;
        end
```

먼저 Load-use data hazard를 핸들링하기 위한 스톨 구현이다. 조건문은 현재 EX 단계에 있는 명령어(바로 앞 명령어)가 LOAD 명령어인지(`ID_EX_MemRead`), 앞선 LOAD 명령어가 값을 쓸 레지스터(`ID_EX_rd`)가 현재 ID 단계에 있는 명령어가 읽고자 하는 레지스터(`IF_ID_rs` 또는 `IF_ID_rt`)와 일치하는지 확인하여 조건을 만족하면 컨트롤 신호를 초기화하여 스톨을 건다.

```
                // 2. Branch Data Hazard
        else if (CTRL_BranchEq || CTRL_BranchNeq || CTRL_Ju
mpReg) begin
            if (ID_EX_RegWrite && ID_EX_rd != 0 && (ID_EX_r
d == IF_ID_rs || ID_EX_rd == IF_ID_rt)) begin
                PCWrite = 0; IF_ID_Write = 0; ID_EX_Flush =
1;
            end
            else if (EX_MEM_MemRead && EX_MEM_rd != 0 && (E
X_MEM_rd == IF_ID_rs || EX_MEM_rd == IF_ID_rt)) begin
                PCWrite = 0; IF_ID_Write = 0; ID_EX_Flush =
```

```
1;
            end
            else if (ID_Mispredict) begin
                IF_ID_Flush = 1;
            end
        end
```

Branch data hazard 처리 로직이다.

1. 바로 앞 명령어가 결과를 생성하는 경우(`ID_EX_RegWrite`), 스톨을 걸어준다. ALU 결과든, LOAD 결과든 ID 단계에서는 아직 준비되지 않았기 때문이다.
2. 2 cycle 앞의 명령어가 LOAD일 경우(`EX_MEM_MemRead`), 메모리를 읽기 전이기 때문에 스톨이 필요하다.
3. 스톨이 필요 없는데 branch predictor가 틀렸다면(`ID_Mispredict`) flush시켜준다(`IF_ID_Flush = 1`).

Internal data forwarding 지원 (`RF.v`)

```
        rd_data1 = (RegWrite && wr_addr != 0 && wr_addr ==
rd_addr1) ? wr_data : register_file[rd_addr1];
        rd_data2 = (RegWrite && wr_addr != 0 && wr_addr ==
rd_addr2) ? wr_data : register_file[rd_addr2];
```

Read 로직을 수정하여 read와 write를 동일한 cycle에 할 수 있도록 구현하였다.

BP 모듈 인스턴스화 및 포워딩 Multiplexer 추가 (`CPU.v`)

```
        wire BP_Update_En = (CTRL_BranchEq || CTRL_BranchNe
q || CTRL_Jump) && (IF_ID_Inst != 32'b0);
    BP bp_unit(
        .clk(clk), .rst(rst),
        .PC(PC),
        .Pred_Taken(Pred_Taken), .Pred_Target(Pred_Target),
        .Update_En(BP_Update_En),
        .Update_PC(IF_ID_PC_plus4 - 4),
        .Update_Target(CTRL_Jump ? JumpTarget : BranchTarge
```

```
t),
        .Update_Taken(CTRL_Jump ? 1'b1 : BranchTaken),
        .Update_IsBranch(CTRL_BranchEq || CTRL_BranchNeq),
        .Update_IsJump(CTRL_Jump)
    );
```

BP 모듈을 instantiate하였다.

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

```
        // Forwarding Unit Logic
    wire [1:0] ForwardA = (EX_MEM_RegWrite && EX_MEM_DestRe
g != 0 && EX_MEM_DestReg == ID_EX_rs) ? 2'b10 :
                          (MEM_WB_RegWrite && MEM_WB_DestRe
g != 0 && MEM_WB_DestReg == ID_EX_rs) ? 2'b01 : 2'b00;

    wire [1:0] ForwardB = (EX_MEM_RegWrite && EX_MEM_DestRe
g != 0 && EX_MEM_DestReg == ID_EX_rt) ? 2'b10 :
                          (MEM_WB_RegWrite && MEM_WB_DestRe
g != 0 && MEM_WB_DestReg == ID_EX_rt) ? 2'b01 : 2'b00;

    wire [31:0] forwardA_data = (ForwardA == 2'b10) ? EX_ME
M_ALUResult :
                                (ForwardA == 2'b01) ? MEM_W
B_WriteData : ID_EX_ReadData1;
```

```
    wire [31:0] forwardB_data = (ForwardB == 2'b10) ? EX_ME
M_ALUResult :
                                (ForwardB == 2'b01) ? MEM_W
B_WriteData : ID_EX_ReadData2;

    wire [31:0] ALU_operand1 = ID_EX_ALUSrcA ? forwardA_dat
a : ID_EX_PC_plus4;
    wire [31:0] ALU_operand2 = (ID_EX_ALUSrcB == 2'b00) ? f
orwardB_data :
                               (ID_EX_ALUSrcB == 2'b10) ? I
D_EX_SignExtImm : 32'd0;
```

표에 정리된 데이터 포워딩 로직을 multiplexer가 반영하고 포워딩할 데이터를 로직에 맞게 선택하도록 설계하였다.

```
        wire [31:0] ID_rs_data = (rs == 0) ? 32'd0 :
                                 (EX_MEM_RegWrite && EX_MEM_Des
tReg == rs) ? EX_MEM_ALUResult : rd_data1;
    wire [31:0] ID_rt_data = (rt == 0) ? 32'd0 :
                             (EX_MEM_RegWrite && EX_MEM_Des
tReg == rt) ? EX_MEM_ALUResult : rd_data2;
```

Branch resolution을 위해 ID 단계로 EX 단계의 결과를 포워딩하였다.

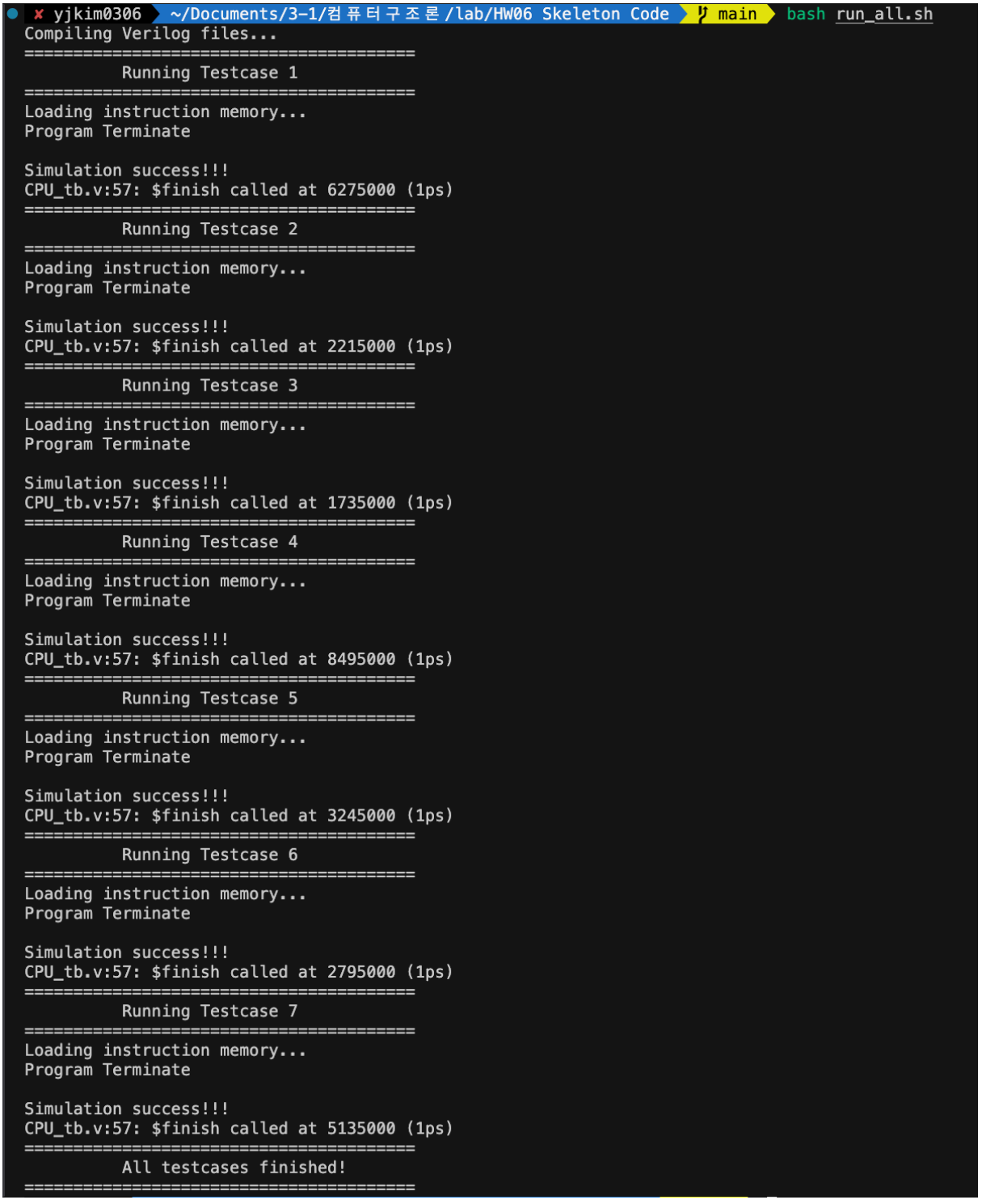
```
    wire [31:0] Actual_Next_PC = CTRL_JumpReg ? ID_rs_data
:
                                 CTRL_Jump ? JumpTarget :
                                 BranchTaken ? BranchTarget
: IF_ID_PC_plus4;

    wire ID_IsBranchOrJump = CTRL_BranchEq || CTRL_BranchNe
q || CTRL_Jump || CTRL_JumpReg;
    wire ID_Mispredict = ID_IsBranchOrJump && (Actual_Next_
PC != IF_ID_Pred_PC) && (IF_ID_Inst != 32'b0);

    assign next_PC = ID_Mispredict ? Actual_Next_PC :
                     Pred_Taken ? Pred_Target : PC_plus4;
```

Branch misprediction일 경우를 대비하여 실제 nextPC값(`Actual_Next_PC`)을 ID 단계에서 올바르게 계산하고, misprediction 여부를 예측값과 이 실제값을 비교하여 판단한 뒤, misprediction일 경우(`ID_Mispredict`) `Actual_Next_PC`를 `next_PC`에 할당한다.

테스트 결과



7개의 테스트케이스 모두 통과하였다.